API & Module Reference

ChequePrint Pro — Internal Module API

Version: 1.0.0

1. Electron IPC API (preload.js exposed API)

All methods are available via `window.electronAPI` in the renderer process.

`electronAPI.printCheque(payload)`

Dispatches a single cheque print job to the OS printer.

Parameters:

```
interface PrintChequePayload {
  bankCode: string;        // e.g., "SBI", "HDFC"
  fields: {
    payee_name: string;
    amount: number;
    amount_words: string;
    cheque_date: string;   // "DD/MM/YYYY"
    date_dd?: string;
    date_mm?: string;
    date_yyyy?: string;
    account_ref?: string;
  };
  offsets?: {
    x: number;             // mm, default 0
    y: number;             // mm, default 0
  };
  printerName?: string;    // Uses default printer if omitted
  dpi?: number;            // Default: 300
}
```

Returns: `Promise<{ success: boolean; jobId?: string; error?: string }>`

`electronAPI.printBatch(payload)`

Dispatches multiple cheque print jobs sequentially.

Parameters:

```
interface PrintBatchPayload {
  bankCode: string;
  cheques: PrintChequePayload['fields'][];
  offsets?: { x: number; y: number };
  printerName?: string;
  batchId: string;        // UUID for grouping records
}
```

Returns: `Promise<{ success: boolean; printed: number; failed: number; errors: string[] }>`

`electronAPI.printTestPage(bankCode, printerName?)`

Prints a grid calibration test page on plain A4 paper.

Returns: `Promise<{ success: boolean }>`

`electronAPI.saveRecord(record)`

Saves a cheque record to the SQLite database.

Parameters:

```
interface ChequeRecord {
  cheque_number?: string;
  bank_code: string;
  payee_name: string;
  amount: number;
  amount_words: string;
  cheque_date: string;
  account_ref?: string;
  ifsc_code?: string;
  narration?: string;
  status?: 'printed' | 'pending' | 'cancelled';
  batch_id?: string;
}
```

Returns: `Promise<{ success: boolean; id: number }>`

`electronAPI.queryRecords(filters)`

Retrieves cheque records with optional filters.

Parameters:

```
interface RecordFilters {
  bankCode?: string;
  payee?: string;         // Partial match
```

```
  dateFrom?: string;       // "DD/MM/YYYY"
  dateTo?: string;         // "DD/MM/YYYY"
  status?: 'printed' | 'pending' | 'cancelled';
  limit?: number;          // Default: 100
  offset?: number;         // For pagination
}
```

Returns: `Promise<{ records: ChequeRecord[]; total: number }>`

`electronAPI.exportRecords(filters, filePath)`

Exports filtered records as a CSV file.

Returns: `Promise<{ success: boolean; rowsExported: number }>`

`electronAPI.getSetting(key)`

Reads a single application setting.

Returns: `Promise<string | null>`

`electronAPI.setSetting(key, value)`

Writes a single application setting.

Returns: `Promise<{ success: boolean }>`

`electronAPI.getAvailablePrinters()`

Lists all printers installed on the operating system.

Returns: `Promise<{ name: string; isDefault: boolean; status: string }[]>`

`electronAPI.getTemplates()`

Returns all bank templates (both seeded and custom).

Returns: `Promise<BankTemplate[]>`

`electronAPI.saveCalibration(templateId, printerName, offsetX, offsetY)`

Saves printer calibration offset for a specific bank template and printer.

Returns: `Promise<{ success: boolean }>`

`electronAPI.getCalibration(templateId, printerName)`

Retrieves calibration offset.

Returns: `Promise<{ offset_x_mm: number; offset_y_mm: number } | null>`

2. numberToWords Module

```
// src/lib/numberToWords.ts

/**
 * Converts a numeric amount to Indian words format.
 * Handles values from 0 to 999,999,999.
 *
 * @param amount - Numeric amount (integer or decimal)
 * @returns Amount in words with "Only" suffix
 *
 * @example
 * numberToIndianWords(125000) // "One Lakh Twenty Five Thousand Only"
 * numberToIndianWords(75050)  // "Seventy Five Thousand And Fifty Only"
 * numberToIndianWords(5500.50) // "Five Thousand Five Hundred And Paise Fifty Only"
 */
export function numberToIndianWords(amount: number): string;

/**
 * Formats a number using Indian number system (with lakh/crore separators).
 * @example
 * formatIndianNumber(1250000) // "12,50,000"
 */
export function formatIndianNumber(amount: number): string;
```

3. Validation Module

```
// src/lib/validation.ts
import { z } from 'zod';

export const chequeSchema: z.ZodObject<...>;

/**
 * Validates a single cheque data object.
 * Returns parsed data if valid, throws ZodError if invalid.
 */
export function validateCheque(data: unknown): ChequeFormData;

/**
 * Validates a batch of cheque rows (from CSV import).
 * Returns array of { row, valid, errors } objects.
 */
export function validateBatch(rows: unknown[]): BatchValidationResult[];
```

4. CSV/Excel Import Module

```
// src/lib/csvImport.ts

/**
 * Parses a CSV file and returns validated cheque rows.
 * @param filePath - Absolute path to the CSV file
 */
export async function importCSV(filePath: string): Promise<BatchValidationResult[]>

/**
 * Parses an Excel (.xlsx) file and returns validated cheque rows.
 * Uses first sheet by default.
 * @param filePath - Absolute path to the XLSX file
 */
export async function importExcel(filePath: string): Promise<BatchValidationResult[]

/**
 * Maps CSV column headers to cheque field names.
 * Supports common header aliases (e.g., "Payee" → "payee_name").
 */
export function detectColumnMapping(headers: string[]): Record<string, string>;
```

5. Store (Zustand)

```
// src/store/appStore.ts
interface AppStore {
  // State
  selectedBank: string;
  chequeData: ChequeFormData;
  records: ChequeRecord[];
  batchQueue: ChequeFormData[];
  settings: Record<string, string>;
  isLoading: boolean;
  lastError: string | null;

  // Actions
  setBank: (bankCode: string) => void;
  updateChequeField: (field: keyof ChequeFormData, value: string | number) => void;
  resetForm: () => void;
  loadRecords: (filters?: RecordFilters) => Promise<void>;
  addToBatch: (cheque: ChequeFormData) => void;
  clearBatch: () => void;
  loadSettings: () => Promise<void>;
  updateSetting: (key: string, value: string) => Promise<void>;
}
```

6. Error Codes Reference

Code	Description	Resolution
`PRINT_001`	No printer found	Install and configure a printer in OS settings
`PRINT_002`	Custom paper size not supported	Manually set paper size in printer preferences
`PRINT_003`	Print job timed out	Check printer is online and not in error state
`DB_001`	Database file locked	Close other instances of ChequePrint Pro
`DB_002`	Database corruption detected	Restore from backup via Settings > Restore
`IMPORT_001`	CSV parse error	Check CSV encoding (must be UTF-8)
`IMPORT_002`	Required column missing	Ensure CSV has `payee_name`, `amount`, `date` columns
`TEMPLATE_001`	Bank template not found	Select a valid bank from the dropdown
`VALIDATION_001`	Amount out of range	Amount must be between ₹1 and ₹99,99,99,999
`VALIDATION_002`	Stale cheque date	Date is more than 3 months in the past